Q1 Constructor and Destructor Execution Order

Consider the following program:

```
class Demo {
int x;
public:
Demo(int a) { x = a; cout << "Constructor " << x << endl; }
~Demo() { cout << "Destructor " << x << endl; }
};

int main() {
Demo d1(10), d2(20);
}
```

Tasks:

1. Predict the exact output sequence.
2. Analyze the order of constructor and destructor execution.
3. Explain why destructors are called in reverse order.

Submitted Files

1924260003D_q15475_co3_at1_analytical_problem_solving_1.pdf — View

Awaiting faculty evaluation

Q2 Constructor Overloading Resolution

10 marks

```
class Product {
public:
Product() { cout << "Default"; }
Product(int x) { cout << "One param"; }
Product(int x, int y) { cout << "Two param"; }
};
```

Tasks:

1. Analyze which constructor is called for different object declarations.
2. Predict the output for Product p1; Product p2(10); Product p3(10,20);
3. Explain how the compiler resolves constructor overloading.

Submitted Files

1924260003D_q15476_co3_at1_analytical_problem_solving_1.pdf — View

Awaiting faculty evaluation

Q3 Operator Overloading for Complex Numbers

10 marks

```
class Complex {
int r, i;
public:
Complex(int a, int b) { r = a; i = b; }
Complex operator+(Complex c) {
return Complex(r + c.r, i + c.i);
}
};
```

Tasks:

1. Analyze how the + operator works for objects.
2. Predict the result of adding two Complex objects.
3. Compare operator overloading with function-based addition.

Submitted Files

1924260003D_q15477_co3_at1_analytical_problem_solving_1.pdf — View

Awaiting faculty evaluation

Q4 Shallow Copy in Hospital Management System

A hospital management system maintains patient records where each Patient object stores dynamically allocated medical history details (using pointers). When patient records are copied for insurance processing or referral to another hospital, the system uses copy constructors.

Tasks:

1. What problems may occur if the system uses a shallow copy while duplicating patient records?
2. How can this lead to issues such as data corruption or accidental modification of medical history?
3. Explain how a deep copy mechanism ensures safe duplication of patient records.

Submitted Files

1924260003D_q15478_co3_at1_analytical_problem_solving_1.pdf — View

Awaiting faculty evaluation

Q5 Polynomial Class with Operator Overloading

A polynomial can be represented as a list of coefficients, where each index represents the power of the variable (e.g., $3 + 2x + 5x^2 \rightarrow [3, 2, 5]$). Design a C++ class Polynomial that stores coefficients dynamically and overloads the + operator to add two polynomials.

Tasks:

1. Analyze how operator overloading simplifies polynomial addition compared to a traditional function-based implementation.
2. Examine the internal working of adding two polynomials of different degrees.
3. Discuss memory handling issues when using dynamic arrays for storing coefficients.

Submitted Files

1924260003D_q15479_co3_at1_analytical_problem_solving_1.pdf

View

Awaiting faculty evaluation

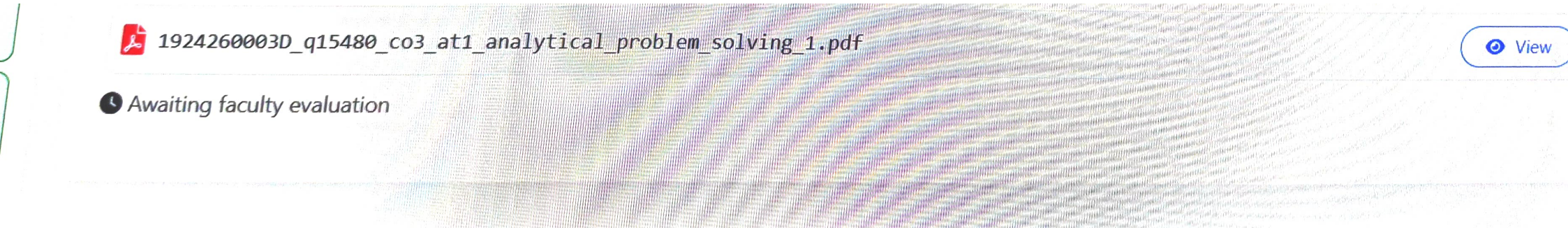
em Solving

Q6 Static Member Behavior in a Banking System

```
class Account {
static int totalAccounts;
int accNo;
public:
Account() { totalAccounts++; accNo = totalAccounts; }
static int getTotal() { return totalAccounts; }
};
int Account::totalAccounts = 0;

int main() {
Account a1, a2, a3;
cout << Account::getTotal();
{
Account a4;
cout << Account::getTotal();
}
cout << Account::getTotal();
}
```

Tasks:


1924260003D_q15480_co3_at1_analytical_problem_solving_1.pdf
View
Awaiting faculty evaluation

Q7 Friend Function vs Member Function

```
class Distance {
int meters;
public:
Distance(int m) { meters = m; }
friend Distance add(Distance, Distance);
};

Distance add(Distance d1, Distance d2) {
return Distance(d1.meters + d2.meters);
}
```

Tasks:

1. Analyze how the friend function accesses private members of the class.
2. Compare the use of a friend function with a member function for the same operation.
3. Justify when a friend function is more appropriate than a member function, and discuss its impact on encapsulation.

Submitted Files

1924260003D_q15481_co3_at1_analytical_problem_solving_1.pdf

Awaiting faculty evaluation

em Solving

Q8 Inheritance and Constructor Chaining

```
class A {
public:
A() { cout << "A "; }
 ~A() { cout << "~A "; }
 };

 class B : public A {
 public:
 B() { cout << "B "; }
  ~B() { cout << "~B "; }
  };

  class C : public B {
  public:
  C() { cout << "C "; }
  ~C() { cout << "~C "; }
  };
  int main() {
  C obj;
```

1924260003D_q15482_co3_at1_analytical_problem_solving_1.pdf

View

Awaiting faculty evaluation

em Solving

Q9 Virtual Functions and Runtime Polymorphism

A graphics application uses a base class Shape and derived classes Circle, Rectangle, and Triangle. Each shape needs to compute its own area, but the application stores all shapes in a single array of Shape* pointers and invokes area() on each.

Tasks:

1. Analyze why declaring area() as a virtual function in the base class is essential.
2. Predict what would happen if area() were not declared virtual when called through a base class pointer.
3. Justify how runtime polymorphism enables extensibility, and discuss the trade-off in performance due to the vtable mechanism.

Submitted Files

1924260003D_q15483_co3_at1_analytical_problem_solving_1.pdf View

Awaiting faculty evaluation

Q10 Exception Handling in a File Processing System

A file processing system reads student records from multiple files. Some files may not exist, some may contain invalid data, and some may be empty. The developer wants to ensure that the program does not crash and continues processing the remaining files gracefully.

Tasks:

1. Analyze how try–catch blocks can be used to handle different types of exceptions (file not found, invalid format, empty file).
2. Examine the role of custom exception classes in providing meaningful error messages.
3. Justify why exception handling is preferred over traditional error-checking with return codes in large object-oriented systems.

Submitted Files

1924260003D_q15484_co3_at1_analytical_problem_solving_1.pdf — View

Awaiting faculty evaluation